

Investigación Tkinter

1.- Que librerías incluye Python para Interfaces Graficas de Usuario?

Tkinter

La biblioteca se utiliza para crear aplicaciones GUI para computadoras y no es compatible con el desarrollo de aplicaciones móviles. El framework Tkinter es bueno para aquellos que son nuevos en Python y quieren aprender a desarrollar aplicaciones.

Kivy

Este es el marco de interfaz gráfica de usuario de Python más preferido sobre el que se pueden construir aplicaciones móviles y de computadora. Es un marco Open GL y viene en forma de una biblioteca de terceros fabricada por la comunidad de Kivy.

Esta biblioteca contiene varias funciones que pueden ayudar a los desarrolladores a crear aplicaciones sólidas para su sistema. Los sistemas operativos compatibles con esta biblioteca son Windows, Mac, Linux, Android, ios y Raspberry Pi. Es de ejecución muy rápida y contiene widgets interesantes que pueden hacer que la aplicación se vea más hermosa.

PyQT

PyQT es una increíble biblioteca de Python que se utiliza para crear aplicaciones multiplataforma tanto para computadoras como para dispositivos móviles y se ejecuta en el marco Qt desarrollado por Nokia. Esta biblioteca viene en dos versiones que son de pago y gratuitas.

El principal inconveniente de la versión gratuita es la limitación de las funciones que ofrece, pero sí, es adecuada para que un principiante comience con la versión gratuita. Los sistemas operativos que admite son Windows, Mac, Linux, Android y Zaurus.

PySide

Esta es otra poderosa herramienta de desarrollo de GUI de Python que viene vinculada con el marco de Qt al igual que PyQt y ayuda a los desarrolladores a crear aplicaciones multiplataforma con facilidad.

Esto también tiene una característica especial: se combina fácilmente con PyQt 4 y, por lo tanto, ayuda a los desarrolladores a cambiar de PyQt a PySide para crear mejores aplicaciones. Los sistemas operativos compatibles con esta biblioteca son Windows, Mac, Linux, Android y Maemo.

PySimpleGUI

PySimpleGUI envuelve la totalidad de Tkinter, que viene con Python. PySimpleGUI ha envuelto la mayor parte de PySide2, pero solo una pequeña parte de wxPython.

2.- Que es Tkinter?

Tkinter es la interfaz de Python para el kit de herramientas Tk, que es una biblioteca GUI multiplataforma. Tkinter proporciona una forma rápida y fácil de crear aplicaciones GUI en Python, ya que incluye una variedad de controles (widgets) como botones, etiquetas, cuadros de texto y más.

3.- Que es un widget?

Un widget en el contexto de Tkinter (y GUIs en general) es un elemento de la interfaz de usuario que el usuario puede interactuar con o visualizar. Ejemplos de widgets incluyen botones, etiquetas, cuadros de texto, listas desplegables, etc. Cada widget tiene propiedades y métodos específicos que permiten configurar su apariencia y comportamiento.

4.- Como funcionan los siguientes controles:

Frame

- **TK (Frame):** Un Frame es un contenedor en Tkinter que se utiliza para agrupar y organizar otros widgets dentro de una ventana. Los Frame son especialmente útiles cuando necesitas organizar widgets en secciones distintas de tu GUI.

```
python
import tkinter as tk

root = tk.Tk()
frame = tk.Frame(root, width=200, height=200, bg="blue")
frame.pack() # Se puede especificar el lado (top, bottom, left, right) si es necesario
root.mainloop()
```

Button

- **Button:** Un Button es un widget que el usuario puede presionar para ejecutar una acción específica, como llamar a una función.

```
python
import tkinter as tk

def on_button_click():
    print("Button clicked")

root = tk.Tk()
```

```
button = tk.Button(root, text="Click me", command=on_button_click)
button.pack()
root.mainloop()
```

Label

- **Label:** Una Label se usa para mostrar texto o imágenes estáticas en la interfaz. Es útil para mostrar información al usuario.

```
python
import tkinter as tk

root = tk.Tk()
label = tk.Label(root, text="Hello, Tkinter!")
label.pack()
root.mainloop()
```

Spinbox

- **Spinbox:** Un Spinbox permite al usuario seleccionar un valor de una secuencia de valores predefinidos. Es útil para seleccionar números o valores de un rango específico.

```
python
import tkinter as tk

root = tk.Tk()
spinbox = tk.Spinbox(root, from_=0, to=10)
spinbox.pack()
root.mainloop()
```

Checkbox

- **Checkbox:** Un Checkbutton (checkbox) es un botón de opción que puede estar seleccionado o no. Se utiliza para permitir al usuario hacer selecciones múltiples.

```
python
import tkinter as tk

root = tk.Tk()
checkbox = tk.Checkbutton(root, text="Check me")
checkbox.pack()
root.mainloop()
```

Text

- **Text:** Un widget de Text permite al usuario ingresar y editar texto en múltiples líneas. Es útil para campos de texto grandes como comentarios o descripciones.

```
python
import tkinter as tk

root = tk.Tk()
text = tk.Text(root, height=5, width=30)
text.pack()
root.mainloop()
```

RadioButton

- **RadioButton:** Un Radiobutton es un botón de opción que permite al usuario seleccionar una opción de un conjunto de opciones mutuamente excluyentes.

```
python
import tkinter as tk

root = tk.Tk()
radio1 = tk.Radiobutton(root, text="Option 1", value=1)
radio2 = tk.Radiobutton(root, text="Option 2", value=2)
radio1.pack()
radio2.pack()
root.mainloop()
```

Image

- **Image:** El widget Label también puede usarse para mostrar imágenes. Para ello, se usa un objeto PhotoImage o BitmapImage.

```
python
import tkinter as tk

root = tk.Tk()
img = tk.PhotoImage(file="path/to/image.png")
label = tk.Label(root, image=img)
label.pack()
root.mainloop()
```

5.- Como funcionan los siguientes gestores de geometría de componentes:

Pack

- **Pack:** El gestor de geometría pack organiza los widgets en bloques antes o después de otros widgets. Se pueden especificar parámetros como side, fill, y expand para controlar su disposición.

```
python
import tkinter as tk

root = tk.Tk()
label1 = tk.Label(root, text="Top")
label1.pack(side=tk.TOP)
label2 = tk.Label(root, text="Bottom")
label2.pack(side=tk.BOTTOM)
root.mainloop()
```

Place

- **Place:** El gestor de geometría place permite una disposición absoluta de los widgets, especificando las coordenadas exactas x e y.

```
python
import tkinter as tk

root = tk.Tk()
label = tk.Label(root, text="Placed label")
label.place(x=50, y=100)
root.mainloop()
```

Grid

- **Grid:** El gestor de geometría grid organiza los widgets en una cuadrícula utilizando filas y columnas. Es especialmente útil para interfaces más complejas.

```
python
import tkinter as tk

root = tk.Tk()
label1 = tk.Label(root, text="Row 0, Column 0")
label1.grid(row=0, column=0)
label2 = tk.Label(root, text="Row 0, Column 1")
label2.grid(row=0, column=1)
label3 = tk.Label(root, text="Row 1, Column 0")
label3.grid(row=1, column=0)
root.mainloop()
```



Referencias:

Facialix. (2020, 14 septiembre). Las mejores bibliotecas de código para crear interfaces gráficas en Python. Facialix. <https://blog.facialix.com/las-mejores-bibliotecas-de-codigo-para-crear-interfaces-graficas-en-python/>

tkinter.ttk — *Tk widgets temáticos*. (s. f.). Python Documentation.

<https://docs.python.org/es/3.11/library/tkinter.ttk.html#ttk-widgets>